

Algorithms

Sorting Algorithms

- Sorting algorithms are **methods used to arrange a set of data into a specific order.**
- Orders such as **numerical or alphabetical** order, and can be in **ascending or descending** order.
- They are fundamental in computer science, **enabling efficient organization and retrieval of data**, and are **crucial for optimizing other algorithms** that require sorted input.

Algorithms

Sorting Algorithms

Key Considerations:

- **Time Complexity:** How the algorithm's runtime scales with the size of the input (e.g., $O(n^2)$, $O(n \log n)$).
- **Space Complexity:** How much extra memory the algorithm requires.
- **Stability:** Whether the algorithm preserves the relative order of equal elements.
- **Use Cases:** Different algorithms are better suited for different types of data and applications

Algorithms

Sorting Algorithms

Why are sorting algorithms important?

- **Data organization:** Sorting makes it easier to search, filter, and analyze data.
- **Algorithm optimization:** Many algorithms, like search algorithms, perform better on sorted data.
- **Data representation:** Sorting can be used to present data in a human-readable format.

Algorithms

Sorting Algorithms

Types of Sorting Algorithms:

- **Comparison-based:** These algorithms compare elements to determine their relative order.
- **Non-comparison-based:** These algorithms don't rely on comparing elements directly.

Algorithms

Sorting Algorithms

Comparison-based: These algorithms compare elements to determine their relative order. Examples include:

- **Bubble Sort:** Simple, but inefficient for large datasets.
- **Selection Sort:** Finds the minimum element and swaps it with the first element, repeating for the remaining unsorted portion.
- **Insertion Sort:** Similar to how you might sort playing cards, by inserting each element into its correct position in the sorted portion.
- **Merge Sort:** Divides the list into smaller sublists, sorts them, and then merges them back together.
- **Quick Sort:** Uses a "pivot" element to partition the list, and recursively sorts the partitions.
- **Heap Sort:** Uses a heap data structure for efficient sorting.

Algorithms

Sorting Algorithms

1. Bubble Sort – Visual Explanation

Concept: Compare adjacent elements and swap them if they are in the wrong order. Repeat until sorted.

Visualization Example: Initial array: `[5, 2, 4, 3, 1]`

Pass 1:

css

 Copy

 Edit

`[5, 2, 4, 3, 1] → [2, 5, 4, 3, 1]`

`[2, 5, 4, 3, 1] → [2, 4, 5, 3, 1]`

`[2, 4, 5, 3, 1] → [2, 4, 3, 5, 1]`

`[2, 4, 3, 5, 1] → [2, 4, 3, 1, 5] ✓`

After each pass, the largest number "bubbles up" to the right.

Algorithms

Sorting Algorithms

2. Selection Sort – Visual Explanation

Concept: Find the smallest element and swap it with the first unsorted element.

Visualization Example: Initial array: `[5, 2, 4, 3, 1]`

Step by step:

CSS

 Copy

 Edit

Select min from `[5, 2, 4, 3, 1]` → Swap 5 and 1 → `[1, 2, 4, 3, 5]`

Select min from `[2, 4, 3, 5]` → No change

Select min from `[4, 3, 5]` → Swap 4 and 3 → `[1, 2, 3, 4, 5]`

Algorithms

Sorting Algorithms

3. Insertion Sort – Visual Explanation

Concept: Build the sorted list one element at a time by inserting it into its correct position.

Visualization Example: Initial array: `[5, 2, 4, 3, 1]`

CSS

 Copy

 Edit

`[5]` → already sorted

Insert `2` → `[2, 5]`

Insert `4` → `[2, 4, 5]`

Insert `3` → `[2, 3, 4, 5]`

Insert `1` → `[1, 2, 3, 4, 5]`

Algorithms

Sorting Algorithms

⚡ 4. Quick Sort – Visual Explanation

Concept: Pick a pivot element and partition the array into elements less than and greater than the pivot.
Recursively sort the subarrays.

Visualization Example: Initial array: [5, 3, 8, 4, 2, 6, 1, 7, 7]

Pick pivot 7 → Partition:

[5, 3, 4, 2, 6, 1] | 7 | [8, 7]

Left: Quick sort [5, 3, 4, 2, 6, 1] → [1, 2, 3, 4, 5, 6]

Right: Quick sort [8, 7] → [7, 8]

Final: [1, 2, 3, 4, 5, 6] + [7] + [7, 8] = [1, 2, 3, 4, 5, 6, 7, 7, 8]

https://www.youtube.com/watch?v=MZaf_9lZCrc – Quick sort in 4mins

Algorithms

Sorting Algorithms

5. Merge Sort – Visual Explanation

Concept: Use divide-and-conquer to split the array into halves, sort each half, and merge them.

Visualization Example: Initial array: `[5, 2, 4, 7, 1, 3, 6]`

Split: `[5, 2, 4]` and `[7, 1, 3, 6]`

→ `[5]` + `[2, 4]` → `[2, 4, 5]`

→ `[7]` + `[1, 3, 6]` → `[1, 3, 6, 7]`

Merge: `[2, 4, 5]` + `[1, 3, 6, 7]` → `[1, 2, 3, 4, 5, 6, 7]`

Algorithms

Sorting Algorithms

6. Heap Sort – Visual Explanation

Concept: Turn the array into a max heap, then extract the max one by one and rebuild the heap.

Visualization Example: Initial array: `[4, 10, 3, 5, 1]`

Build Max Heap → `[10, 5, 3, 4, 1]`

Extract max (10), move to end → `[1, 5, 3, 4] + [10]`

Reheapify → `[5, 4, 3, 1]`

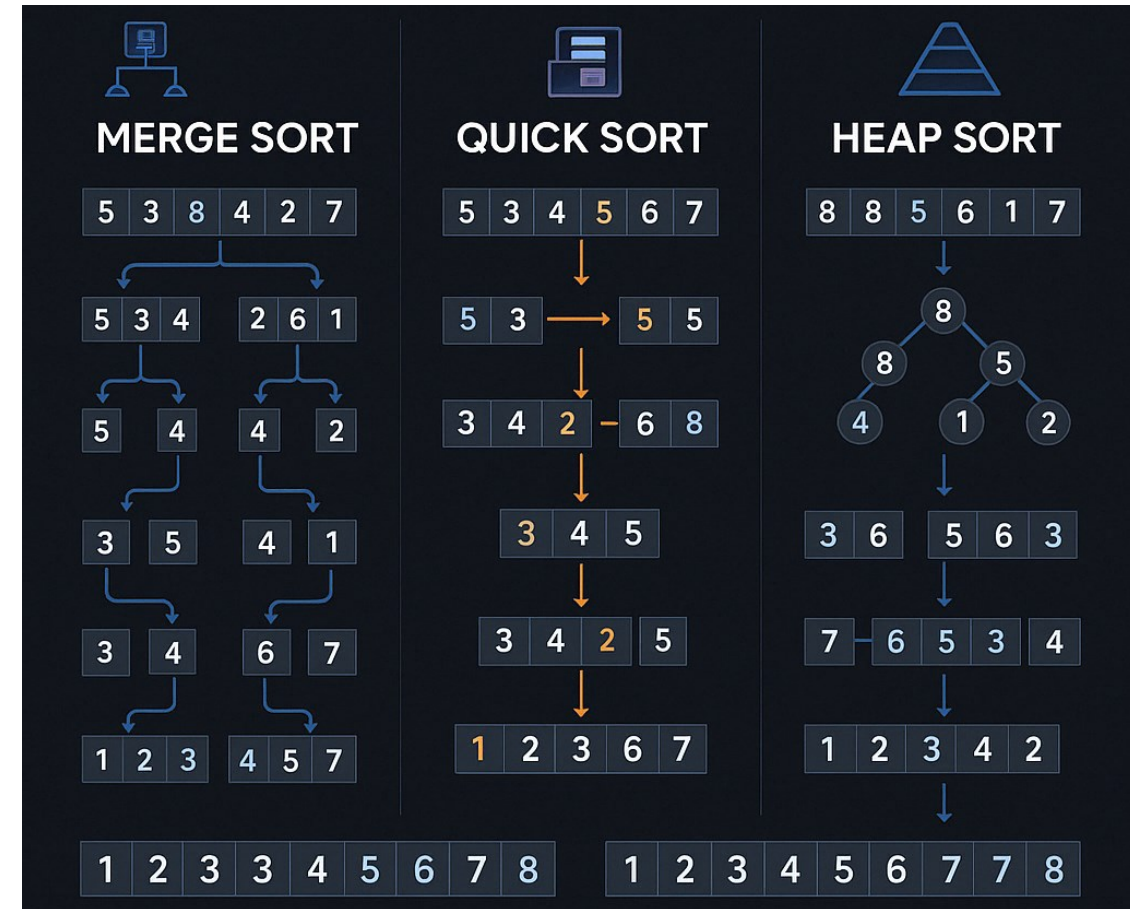
Repeat until sorted → `[1, 3, 4, 5, 10]`

<https://www.youtube.com/watch?v=Q1yi1eaqN7I> – Heap Sort in 4 Mins

Algorithms

Sorting Algorithms

- **Merge Sort:** Uses divide and conquer; splits arrays and merges them in order. **Time Complexity:** $O(n \log n)$ - **Stable Sort**
- **Quick Sort:** Picks a pivot, partitions array around it, recursively sorts. **Average Time:** $O(n \log n)$. **Worst Case:** $O(n^2)$ (but rare with good pivots)
- **Heap Sort:** Builds a heap and extracts the max/min element. **Time Complexity:** $O(n \log n)$. **In-place, not stable.**



Algorithms

Sorting Algorithms

Non-comparison-based: These algorithms don't rely on comparing elements directly. Examples include:

- **Counting Sort:** Suitable for sorting integers within a known range.
- **Radix Sort:** Sorts elements based on their individual digits or characters.
- **Bucket Sort:** Distributes elements into buckets and then sorts each bucket.

Algorithms

Sorting Algorithms

7. Counting Sort – Visual Explanation

Concept: Count the number of times each element appears, then reconstruct the array in order.

Visualization Example: Initial array: [4, 2, 2, 8, 3, 3, 1]

```
Count: [0,1,2,2,1,0,0,0,1] (Index = value)
Build output: [1,2,2,3,3,4,8]
```

Best for small range of integers

<https://www.youtube.com/watch?v=OKd534EWcdk> – [Counting Sort in 5 Mins](#)

Algorithms

Sorting Algorithms

8. Radix Sort – Visual Explanation

Concept: Sort numbers digit by digit starting from the least significant digit using a stable sort (e.g., counting sort).

Visualization Example: Initial array: [170, 45, 75, 90, 802, 24, 2, 66]

Sort by units → [170, 90, 802, 2, 24, 45, 75, 66]

Sort by tens → [802, 2, 24, 45, 66, 170, 75, 90]

Sort by hundreds → [2, 24, 45, 66, 75, 90, 170, 802]

<https://www.youtube.com/watch?v=1mh2vilbZMg> – [Radix Sort in 5 Minutes](#)

Algorithms

Sorting Algorithms

□ 9. Bucket Sort – Visual Explanation

Concept: Distribute elements into buckets, sort each bucket, and concatenate the results.

Visualization Example: Initial array: `[0.78, 0.17, 0.39, 0.26, 0.72, 0.94, 0.21, 0.12, 0.23, 0.68]`

Distribute into buckets:

`[0.12, 0.17, 0.21, 0.23, 0.26, 0.39, 0.68, 0.72, 0.78, 0.94]` (after sorting buckets)

Best for uniformly distributed data in $[0, 1)$

<https://www.youtube.com/watch?v=VuXbEb5ywrU> – [Bucket Sort in 2 Mins](#)

Algorithms

Searching Algorithms

- Search algorithms are **methods used to locate specific data** within a larger dataset.
- They are **essential for efficiently retrieving information** from various data structures and systems.
- Some common examples include **linear search, binary search**, and various graph search algorithms like **Breadth-First Search (BFS)** and **Depth-First Search (DFS)**.
- In essence, search algorithms **provide the tools to efficiently find specific items within a dataset**, which is **a fundamental task in computer science** and many other fields.

Algorithms

Searching Algorithms

Here's a more detailed breakdown:

1. Linear Search:

- This is the simplest search algorithm, where each item in the dataset is checked sequentially until the target is found or the end of the dataset is reached.
- It's efficient for small, unsorted datasets but can become slow for large datasets.

2. Binary Search:

- This algorithm works on sorted datasets. It repeatedly divides the dataset in half, comparing the middle element with the target.
- If the target is less than the middle element, the search continues in the lower half; otherwise, it continues in the upper half.
- Binary search is significantly more efficient than linear search for sorted datasets.

Algorithms

Searching Algorithms

3. Graph Search Algorithms:

- **Breadth-First Search (BFS):** Explores all nodes at a given depth level before moving to the next depth level.
- **Depth-First Search (DFS):** Explores a branch as far as possible before backtracking and exploring another branch.

These algorithms are commonly used in artificial intelligence and other areas where exploring connections between data elements is necessary.

Algorithms

Searching Algorithms

4. Other Search Algorithms:

- **Interpolation Search:** An improvement over binary search, especially for uniformly distributed data, as it predicts the position of the target based on its value.
- **Exponential Search:** Searches by doubling the search space until the target is found or the search space is exhausted.
- **Fibonacci Search:** Similar to binary search, but uses Fibonacci numbers to divide the search space.
- **Jump Search:** A variation of linear search that jumps ahead by a fixed step size, reducing search time.